



Personalized Hierarchical Education Agent (LangGraph)

This architecture defines a strict **Orchestrator-Worker** pattern. The Parent Agent acts as the single point of entry and exit, ensuring that every piece of educational content is vetted, personalized, and evaluated before reaching the student.

1. Sub-Agent Definitions & KPIs

Sub-Agent	Technical Role	Success Metric (KPI)
Parent Orchestrator	Supervisor/Router: Interprets user intent and selects the correct graph path.	Routing Accuracy: Correctly identifying "New Concept" vs. "Answer" 98% of the time.
Knowledge Retriever	Context Provider: Connects to Vector DB (RAG) to pull chunked educational data.	Faithfulness: No hallucinations; output must be 100% grounded in retrieved docs.
Personalization Engine	Style Transformer: Maps raw knowledge to the student's interests and reading level.	Alignment: Matching the "Interest Keywords" (e.g., using 'Gaming' terms) in the output.
Concept Evaluator	Diagnostic Analyst: Compares user response against "Ground	Precision: Correctly identifying "Partial Mastery" vs.

	Truth" from the Researcher.	"Misconception."
Profile Updater	Data Sync: Translates natural language evaluation into structured JSON for the DB.	Schema Integrity: Successfully updating the SQL/NoSQL mastery matrix.

2. Memory Architecture & Data Schema

A. Persistent "Long-Term" Memory (Profile DB)

This resides in a relational database (PostgreSQL/Supabase) and is loaded at the start of every session.

- **Student Profile Table:**
 - `learning_style`: (e.g., "Socratic", "Analogy-heavy", "Direct")
 - `interest_graph`: JSON array of keywords used for analogies.
 - `reading_age`: Integer (affects vocabulary complexity).
- **Mastery Matrix Table:**
 - `concept_id`: Unique ID for a curriculum topic.
 - `proficiency_score`: Float (\$0.0\$ to \$1.0\$).
 - `attempts_count`: Number of times the student has engaged with this topic.

B. Ephemeral "Short-Term" Memory (LangGraph State)

The **Graph State** is the shared object passed between agents.

Python

```
class EducationState(TypedDict):
    user_input: str
    student_profile: dict          # Loaded from DB at Start
    retrieved_context: str         # Raw RAG data
    personalized_explanation: str   # Final output text
    evaluation_result: dict        # { "is_correct": bool, "feedback": str }
    internal_monologue: str        # Parent's reasoning for debugging
    active_node: str               # Current sub-agent in control
```

3. Detailed Functional Flows

Flow 1: The "New Knowledge" Loop

1. **Intent Classification:** Parent detects the user wants to learn something new.
2. **RAG Fetch:** Retriever pulls the top 3 most relevant chunks from the Vector DB.
3. **Synthesis:** Personalizer receives the raw chunks + the `interest_graph`. It generates a response where the technical concepts are substituted with relatable analogies.
4. **Active Verification:** Before the Parent sends the message, it forces the **Evaluator** to generate a "Check for Understanding" question based *only* on the new information shared.
5. **Delivery:** User receives the explanation + the question.

Flow 2: The "Correction & Remediation" Loop

1. **Submission:** User answers a question.
2. **Logic Audit:** The **Evaluator** compares the user's answer to the `retrieved_context`.
3. **Gap Analysis:** If wrong, the Evaluator identifies *why* (e.g., "Student understands gravity but confuses it with atmospheric pressure").
4. **Pivot:** Parent sees the "Gap Analysis" and triggers the **Retriever** again, but this time with a specific query: "Differences between gravity and air pressure."
5. **Re-Explanation:** Personalizer delivers a targeted correction.

4. The State Machine Logic (Guardrails)

To ensure the "Parent" never skips a step, the graph utilizes **Directed Edges** and **Conditional Logic Gates**:

The "No-Skip" Policy

- **Gate A (Personalization Check):** If the `personalized_explanation` node returns text that is too long or complex (checked via an LLM judge or heuristic), the edge loops back to the Personalizer for a "Simplification Pass."
- **Gate B (Evaluation Requirement):** The `END` node is unreachable unless the `evaluation_result` object is populated. This forces the system to always think

about the student's progress.

The "Drift" Guardrail

The Parent Agent performs a **Context Check** every 3 turns. It compares the current conversation against the **Mastery Matrix**. If the student is drifting away from the intended learning goal, the Parent is programmed to say: *"That's an interesting point, but let's get back to how this relates to [Original Topic]."*

5. Example Interaction Scenarios

Scenario	Parent Decision	Sub-Agent Chain
User asks a "Why" question	Deep Dive	RAG $\rightarrow$ Personalizer $\rightarrow$ Evaluator
User gives a vague answer	Probe for Clarity	Evaluator (requests more detail) $\rightarrow$ User
User expresses frustration	Emotional Support	Personalizer (shifts tone to Encouraging) $\rightarrow$ RAG (simpler content)
User masters a topic	Progress Update	Evaluator $\rightarrow$ Profile Updater (increments score) $\rightarrow$ Parent (suggests next topic)

6. Technical Integration Requirements

- **Orchestration:** LangGraph (using **StateGraph** for cyclic support).
- **Vector Store:** Metadata filtering is required (e.g., filter by **grade_level** during RAG).
- **Latency Management:** Use **Streaming** for the Personalizer node so the user sees the "Explanation" while the "Evaluator" is still generating the quiz question in the background.

Things not included

1. personalized voice (easy)
2. Subject switching within Runtime (medium)

3. MCP tool integrations for the parent agent sending updates to teachers based on the progress

4. Security guardrail to check every AI response against the **Student's Profile**. It will automatically block or fix any words that might cause distress, trigger past trauma, or make the student feel uncomfortable before they ever see the message